
1 Just-In-Time (JIT) Compiler

- ◆ Converts bytecode into **native machine code** for better performance.
- ◆ Key optimizations:
 - **C1 (Client Compiler)**: Faster startup, used for quick optimizations.
 - **C2 (Server Compiler)**: Aggressive optimizations for long-running apps.
 - **Graal JIT (Java 9+)**: Modern JIT, better for cloud workloads.
- ◆ **Tuning JIT:**

Use `-XX:+PrintCompilation` to monitor JIT optimizations.

2 Garbage Collectors (GC)

- ◆ JVM manages memory via **automatic GC**. Key collectors:
 - **G1GC (Default in Java 9+)** → Low-pause, balanced performance.
 - **ZGC (Java 11+)** → Ultra-low pause (<10ms), scales to TBs.
 - **Shenandoah (Java 12+)** → Low-pause, concurrent GC.
- ◆ **Tuning GC:**

Use `-XX:+UseG1GC`, `-XX:+UseZGC`, and analyze GC logs with `-Xlog:gc`.

3 Profiling & Monitoring Tools

- **JVisualVM** → Live heap, CPU, and thread analysis.
- **JFR (Java Flight Recorder)** → Low-overhead profiling (`-XX:+UnlockCommercialFeatures`).
- **Async Profiler** → Flame graphs for performance tuning.

4 CPU & Memory Usage Analysis

- JVM Flags for Monitoring:

```
-Xms512m -Xmx2g -XX:+UseG1GC -XX:MaxGCPauseMillis=200
```

- Top tools for profiling:
 - `jstat` → Monitor heap usage (`jstat -gc <pid>`).
 - `jmap` → Heap dump (`jmap -dump:live,format=b,file=heap.hprof <pid>`).
 - `jconsole` → Real-time JVM metrics.

5 ThreadLocal & Thread Dumps

- ♦ **ThreadLocal** → Stores per-thread data but risks **memory leaks** if not cleared.
 - ♦ **Thread Dumps** (`jstack <pid>`) → Debug deadlocks & thread states.
-

6 Deadlock Detection

- ◆ Detect with:

```
jstack <pid> | grep "BLOCKED"
```

- ◆ **Prevention Techniques:**

- **Lock ordering** → Always acquire locks in the same sequence.
 - **Timeouts with `tryLock()`** → Avoid indefinite waiting.
 - **Thread dump analysis** → Identify circular dependencies.
-